Casebench — Case Study

Oct 6, 2026 · @sai

TL;DR

Casebench lets you practise a job by doing it: you join a fictional company's Slack-style workspace, work with AI coworkers, query real-looking data, and hand in a write-up graded against what actually happened. It's live at casebench.vercel.app.

Three things make it different from interview-prep sites:

- **The problem isn't handed to you.** Your manager describes a symptom ("watch time is down 21%"); finding the real cause is the job.
- **The answer is checked against a hidden truth, and so is how you got there.** The grader sees the write-up plus a log of every query, message and hint.
- **New problems are researched from real incidents.** An AI author agent reads public postmortems, designs a simulation around one, and proves with SQL that the cause can be found before a human approves it.

The problem

Most interview prep tests whether you know things: a SQL puzzle with one right answer, a multiple-choice quiz, a take-home where the question is fully specified. Real analyst and engineering work rarely looks like that. A manager says a number moved. The data is messy, a coworker has a confident but wrong theory, and part of the job is deciding which question to ask first.

Those skills (scoping a vague problem, ruling out red herrings, asking people the right things, explaining a finding) are what separate a strong hire from a good test-taker, and almost nothing lets you practise them. Casebench is my attempt to build that practice space.

What it is

A run from the solver's side:

1. **Pick a ticket.** The home page looks like an issue tracker. Each problem has a key (CASE-n), a priority, a role and an author.

2. **Read the brief.** It reads like a real ticket: a symptom, some context, and what the manager needs by the end. It never names the cause.
3. **Work in the workspace.** A Slack-style workspace with channels and AI coworkers. Each coworker has a role, a personality and partial knowledge; a teammate may push a plausible but wrong explanation. You query the data files with SQL, message coworkers and can ask for hints.
4. **Submit a write-up.** Root cause, evidence and recommendation, as you would send them to a manager.
5. **Get graded.** An evaluator compares the write-up with the problem's hidden truth and rubric, and also reads the run log (queries, messages, hints) to judge the process, not just the answer.

Anyone can play as a guest. Picking a name creates a profile, so attempts, scores and authored problems show up on a public page at `/u/<name>`. There's no email or password: a one-time profile key lets you sign in on another device.

How it works

Stack: a TypeScript pnpm monorepo. The web app is Next.js 15 (App Router) on Vercel, with Postgres on Neon. AI coworkers and grading run on Google Gemini through an OpenAI-compatible adapter, so swapping providers is a config change.

Packages: `domain` (problem, rubric and run types), `database` (numbered SQL migrations, applied on every deploy), `simulation-engine` (scenario rules), `agents` (coworker personas), `ai` (provider adapter), `content-tools` (problem validation) and `author-agent` (below).

Key decisions:

- **An append-only event log is the source of truth for a run.** Every query, message, hint and submission is an event. Grading reads the log, so process can be scored. Once a run is published, a database trigger freezes it, so a score can't be edited after the fact.
- **The answer never reaches the browser.** Each problem's truth model lives server-side. Coworkers get only what their persona would know, and leak guards rewrite a reply that would give the cause away before the solver has found it.
- **Built to fail gracefully on a free tier.** AI calls retry on rate limits and server errors. Grading falls back to a lighter model, and if a coworker can't answer, the workspace says so instead of hanging.

- **Identity without friction.** Profiles are a name plus a random key. Only a hash of the key is stored, sessions are HMAC-signed cookies, and guest progress merges into the profile when you create one.

The author agent

Hand-writing good simulations is slow, so I built an agent whose only job is to write new ones. Its drafts are credited to a house author, **@cb**. It runs daily on a Vercel cron job and on demand from an admin console.

The pipeline:

1. **Plan.** Pick a role and theme (e.g. latency regression, credential stuffing, metric double-counting) and write search queries.
2. **Research real incidents.** Search Dan Luu's curated list of public postmortems, Hacker News and Wikipedia, with no paid search APIs. Only public HTTPS pages are fetched, everything fetched is treated as untrusted quoted text, and off-topic results are filtered out. With no relevant sources, it re-plans once, then refuses to write anything.
3. **Write a cited brief.** The real pattern, root causes, signals in the data, red herrings and why it's hard. Any citation URL the model invents is dropped.
4. **Design the simulation.** A scenario (brief, coworkers, hidden truth, leak guards), a recipe for synthetic data with the cause "planted" as effects, and SQL checks that should prove the cause.
5. **Generate the data.** A seeded generator turns the recipe into CSV files and a data dictionary.
6. **Quality gate.** The draft must pass:
   - schema validation (e.g. exactly one manager);
   - lint rules, rejecting columns that label the answer (like `is_fraud`) and briefs that give it away;
   - the SQL checks, run in throwaway Postgres temp tables;
   - a **counter-check**: the same checks on data *without* the planted effect. At least one must flip to false, which proves the cause is findable from the data, not just asserted.
7. **Repair.** On failure, the exact errors go back to the model, up to 3 rounds.
8. **Human review.** Passing drafts stay unlisted until an admin approves them at `/admin/agent`. Rejecting deletes the draft.

What broke in production and what I learned

- **The AI went dark after launch.** The Gemini models I'd configured were retired. I moved to current models and added retries, a grading fallback and a visible "coworker unavailable" notice. *Lesson: treat a third-party model like any other dependency that can disappear, and make failure visible.*
- **Author agent, live run 1.** It cited "List of Latin phrases" as research, wrote checks that were always true and added an `is_duplicate` column that handed over the answer. That run produced the relevance filter, the counter-check and the give-away lint. *Lesson: a passing test proves nothing until you've seen it fail.*
- **Run 2.** Malformed leak guards slipped through, and one with no pattern compiled to a regex that matched everything. Fixed with validation plus worked examples in the prompt. *Lesson: models follow examples more reliably than rules.*
- **Run 3.** No relevant research at all, because Hacker News search was too literal. Adding the postmortems corpus and looser matching fixed it. **Run 4** produced a valid draft (an API p99 latency spike after a deploy), now waiting for review.
- **Process.** A stray file swept into an unrelated commit broke CI once. *Lesson: stage files deliberately and keep pull requests to one concern.*

Each of these is written up in the repo's build logs.

Where it stands today

- **Live** at casebench.vercel.app, deployed from `main` on Vercel with Postgres on Neon.
- **Content:** 2 hand-written problems, plus agent drafts going through review.
- **Tests:** 106 unit and database tests and 4 browser end-to-end suites, run in CI on every pull request.
- **Docs:** 13 build logs covering each stage, from the first deploy to the author agent and the UI redesign.
- **Users:** I'm only now sharing it, so there are no usage numbers yet. That's exactly what I'd like your help with.

I built it with Claude Code as an AI pair programmer. I set the product direction and the design decisions, and reviewed, tested and debugged each change in production.

What's next, and how to try it

Try it (about 20 minutes):

1. Open casebench.vercel.app.
2. Pick a ticket and read the brief.
3. Work it in the workspace: query the data and question your coworkers.
4. Submit your write-up and read the feedback.

You can play as a guest, or pick a name to keep your results.

Next on the roadmap:

- Per-user limits on AI usage, so it stays up on a free tier as more people try it.
- More problems across roles (data analysis, engineering, security, operations), mostly from the author agent with human review.
- Better feedback: side-by-side comparison with the intended investigation path.

Feedback is very welcome, especially where a coworker felt unrealistic or the grading felt unfair. Just reply to whoever sent you this.